

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 18 · Testing con JUnit y depuración

JUnit 5: anotaciones @Test, @BeforeEach, @BeforeAll

Assertions: assertEquals, assertTrue, assertThrows, assertSame

Casos límite y calidad de los tests

Depuración: stack traces, breakpoints y cobertura

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 3
Valoración	Pruebas unitarias y depuración

Parte 1: Opción múltiple y Verdadero/Falso

5 puntos — 0,5 cada pregunta

1. ¿Qué anotación de JUnit 5 marca un método como test?
- a) @Test
 - b) @Run
 - c) @TestCase
 - d) @Check

RESPUESTA / SOLUCIÓN

a) @Test — Los métodos anotados con @Test se ejecutan como pruebas unitarias.

-
2. ¿Qué método de Assertions compara un valor esperado con el real?
- a) assertEquals(esperado, real)
 - b) assertSame(esperado, real)
 - c) assertEquals(esperado, real)
 - d) compare(esperado, real)

RESPUESTA / SOLUCIÓN

a) assertEquals(esperado, real) — Usa equals() para comparar; admite un mensaje opcional como tercer argumento.

-
3. ¿Qué anotación ejecuta un método de inicialización antes de CADA test de la clase?
- a) @BeforeEach
 - b) @BeforeAll

- c) @Setup
- d) @Init

RESPUESTA / SOLUCIÓN

a) @BeforeEach — Se ejecuta antes de cada @Test; @BeforeAll lo hace una sola vez antes de todos.

-
4. Los tests de una clase deben ser independientes entre sí: no deben depender del orden de ejecución ni del estado dejado por otro test.

RESPUESTA / SOLUCIÓN

Verdadero — Si un test depende de otro, fallará de forma intermitente; cada test debe preparar su propio estado (p. ej. con @BeforeEach).

-
5. ¿Qué assertion falla cuando la condición que recibe es falsa?

- a) assertTrue(cond)
- b) assertFalse(cond)
- c) assertNull(x)
- d) assertEquals(a, b)

RESPUESTA / SOLUCIÓN

a) `assertTrue(cond)` — Falla si `cond` es `false`; `assertFalse` es la contraria.

6. ¿Cómo se comprueba en JUnit 5 que un método lanza una excepción concreta?

- a) `assertThrows(Tipo.class, () -> metodo())`
- b) try-catch manual siempre
- c) `assertException(Tipo.class)`
- d) `@Expected(Tipo.class)`

RESPUESTA / SOLUCIÓN

a) `assertThrows` — Ejecuta el lambda y verifica que lanza la excepción indicada; además devuelve la excepción para inspeccionarla.

7. ¿Qué assertion comprueba que dos variables apuntan a la MISMA referencia (no equivalentes)?

- a) `assertSame(a, b)`
- b) `assertEquals(a, b)`
- c) `assertIdentical(a, b)`
- d) `assertRef(a, b)`

RESPUESTA / SOLUCIÓN

a) `assertSame(a, b)` — Usa `==`; `assertEquals` usa `equals()`. Para comparar contenido se usa `assertEquals`.

-
8. Un test sin ninguna `assertion` siempre pasa aunque el código probado esté mal.

RESPUESTA / SOLUCIÓN

Verdadero — Si no se aserta nada, JUnit solo comprueba que no haya excepciones; por eso todo test debe verificar resultados con `assertions`.

-
9. ¿Por qué falla `assertEquals(2.5, resultado)` al comparar doubles?

- a) Por la precisión binaria: hay que usar `assertEquals(2.5, resultado, 0.001)`
- b) Porque `double` no se puede comparar
- c) Porque `assertEquals` no acepta `double`
- d) No falla nunca

RESPUESTA / SOLUCIÓN

a) Por la precisión binaria — Los doubles no siempre se representan exactamente; JUnit exige un delta (tercer argumento) en la comparación.

10. ¿Qué anotación ejecuta código una sola vez ANTES de todos los tests de la clase?

- a) @BeforeAll
- b) @BeforeEach
- c) @BeforeClass (JUnit 4)
- d) @SetupAll

RESPUESTA / SOLUCIÓN

- a) @BeforeAll (JUnit 5) — Se ejecuta una vez; el método debe ser static (o de instancia con @TestInstance(PER_CLASS)).
-

Parte 2: Análisis y depuración

3 puntos

1. Este test falla de forma intermitente. ¿Por qué y cómo se corrige? (1,5 pts)

```
import org.junit.jupiter.api.Test;\nimport static org.junit.jupiter.api.Assertions.*;\npublic cla
```

RESPUESTA / SOLUCIÓN

100.0 * 1.21 no es exactamente 121.0 en binario: el resultado real es 120.99999999999999 (o similar).

assertEquals(double, double) sin delta falla por la precisión de coma flotante.

Corrección: assertEquals(121.0, total, 0.001); — el tercer argumento es la tolerancia (delta).

2. El siguiente test debería pasar pero falla. Depura el código de la clase probada. (1,5 pts)

```
public class Util {\n    public static String iniciales(String nombre) {\n        String[] partes
```

RESPUESTA / SOLUCIÓN

El bucle usa `i <= partes.length` en lugar de `i < partes.length`: para "Juan Lopez" hay 2 partes (índices 0 y 1) y el bucle intenta acceder a `partes[2]`.

Eso lanza `ArrayIndexOutOfBoundsException` (o `StringIndexOutOfBoundsException` si el array estuviera vacío), por lo que el test falla.

Corrección: `for (int i = 0; i < partes.length; i++) { res += partes[i].charAt(0); }`

Parte 3: Ejercicios de programación

6 puntos

1. Escribe la clase Calculadora (suma, resta, divide) y su clase de test JUnit 5 que verifique: sumas, restas, división exacta y que dividir entre cero lanza `IllegalArgumentException`. (2 ptos)

RESPUESTA / SOLUCIÓN

```
public class Calculadora {  
    public int suma(int a, int b) { return a + b; }  
    public int resta(int a, int b) { return a - b; }  
    public double divide(double a, double b) {  
        if (b == 0) throw new IllegalArgumentException("División entre cero");  
        return a / b;  
    }  
}  
  
// --- Test ---  
  
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.*;  
  
public class CalculadoraTest {  
    private final Calculadora c = new Calculadora();  
    @Test void sumas() { assertEquals(5, c.suma(2, 3)); }  
    @Test void restas() { assertEquals(-1, c.resta(2, 3)); }  
    @Test void divisionExacta() { assertEquals(2.5, c.divide(5, 2), 0.0001); }  
    @Test void divisionEntreCero() {  
        assertThrows(IllegalArgumentException.class, () -> c.divide(5, 0));  
    }  
}
```

2. Implementa la clase `EsPalindromo` (método estático `esPalindromo(String)`) y su test con casos límite: "ana" (true), "Hola" (false), cadena vacía (true), "Ana" ignorando mayúsculas (true). (2 ptos)

RESPUESTA / SOLUCIÓN

```
public class EsPalindromo {  
    public static boolean esPalindromo(String s) {  
        if (s == null) return false;  
        String limpio = s.toLowerCase().replace(" ", "");  
        return new StringBuilder(limpio).reverse().toString().equals(limpio);  
    }  
}  
  
// --- Test ---  
  
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.*;  
  
public class EsPalindromoTest {  
    @Test void casos() {  
        assertTrue(EsPalindromo.esPalindromo("ana"));  
        assertFalse(EsPalindromo.esPalindromo("hola"));  
        assertTrue(EsPalindromo.esPalindromo(""));  
        assertTrue(EsPalindromo.esPalindromo("Aha"));  
        assertFalse(EsPalindromo.esPalindromo(null));  
    }  
}
```

3. Escribe una clase `ListaTareas` con `@BeforeEach` que inicialice una lista vacía, y tests que verifiquen añadir, eliminar y contar elementos. (2 pts)

RESPUESTA / SOLUCIÓN

```
import java.util.ArrayList;
import java.util.List;

public class ListaTareas {

    private final List<String> tareas = new ArrayList<>();

    public void anadir(String t) { tareas.add(t); }

    public boolean eliminar(String t) { return tareas.remove(t); }

    public int contar() { return tareas.size(); }

}

// --- Test ---

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ListaTareasTest {

    private ListaTareas lista;

    @BeforeEach void setUp() { lista = new ListaTareas(); }

    @Test void anadirIncrementa() {

        lista.anadir("Estudiar");

        assertEquals(1, lista.contar());

    }

    @Test void eliminarFunciona() {

        lista.anadir("Estudiar");

        assertTrue(lista.eliminar("Estudiar"));

        assertEquals(0, lista.contar());

    }

    @Test void eliminarInexistente() {

        assertFalse(lista.eliminar("No existe"));

    }

}
```

Parte 4: Pregunta teórica

2 puntos

1. ¿Qué es la cobertura de código? ¿Un 100 % de cobertura garantiza que el programa no tiene errores? Razona la respuesta.

RESPUESTA / SOLUCIÓN

La cobertura mide el porcentaje de código (líneas, ramas o condiciones) que ejecutan los tests.

Un 100 % de cobertura NO garantiza ausencia de errores: los tests pueden ejecutar el código sin verificar resultados correctos, o no probar valores límite, entradas inesperadas o condiciones de carrera.

La cobertura es una herramienta de calidad útil (detecta código sin probar), pero la clave está en la calidad de las assertions y en probar casos límite.

Plantilla de respuestas

(Páginas en blanco para desarrollar las soluciones)